

# CS471: Introduction to Artificial Intelligence

## Assignment 1: Python (10 points)

Shane Dyrda

9/12/2024

For this problem, you are only allowed to use standard python libraries. You may not use third party libraries or call any shell/bash functions.

You are given a list of tuples of the form ( $\text{float } x$ ,  $\text{float } y$ ,  $\text{float } r$ ) (Let's call these c-tuples). Each c-tuple represents a circle on a rectangular coordinate space, with  $x$  and  $y$  being the coordinates of the center, and  $r$  being the radius. Assume that each c-tuple has a unique radius.

Let a cluster of circles be a group of circles where each circle in the group overlaps with at least one other circle in that group. A path is formed between two circles when they overlap. Define a cluster as a group of  $n$  circles, where each circle is reachable from every other circle through the formed paths.

Write a python script that does the following: Return True if the given circles form a cluster and return false if they don't form a cluster.

Below are some test cases.

### Test case 1:

**Input: [(1, 3, 0.7), (2, 3, 0.4), (3, 3, 0.9)]**

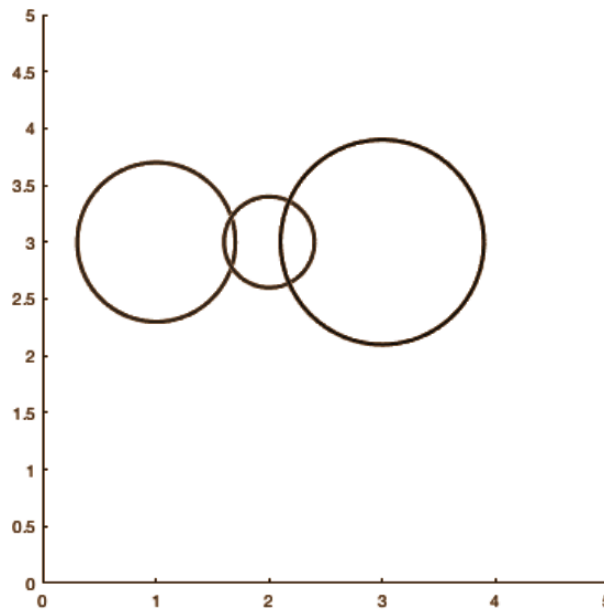


Figure 1: The three circles form the cluster. Output = True

### Test case 2:

**Input: [(1.5, 1.5, 1.3), (4, 4, 0.7)]**

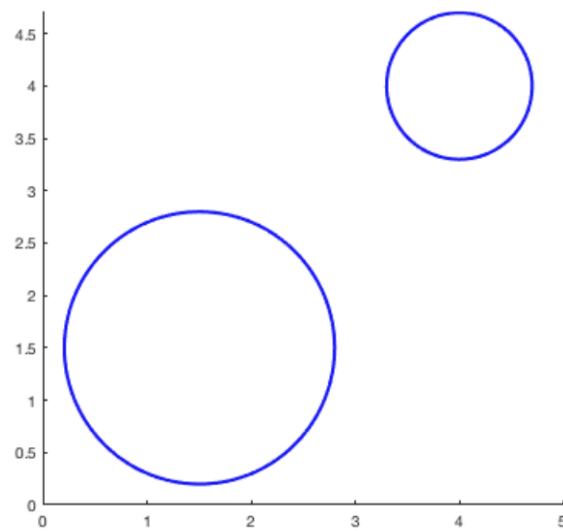


Figure 2: No clusters are found. Output = False

### Test case 3:

**Input: [(0.5, 0.5, 0.5), (1.5, 1.5, 1.1), (0.7, 0.7, 0.4), (4, 4, 0.7)]**

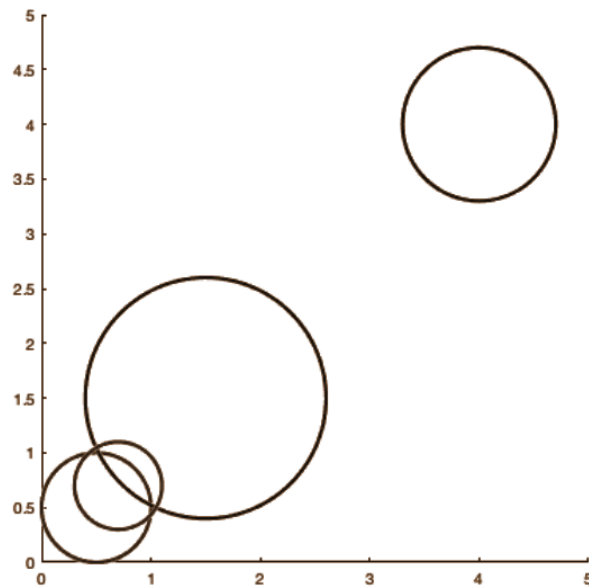


Figure 3: Given circles do not form a cluster. Output = False

#### Test case 4:

$[(0.5, 0.5, 0.5), (1.5, 1.5, 1.1), (0.7, 0.7, 0.4), (4, 4, 0.7),$   
 $(3, 5, 0.8), (3.7, 2, 1.6)]$  **TRUE**

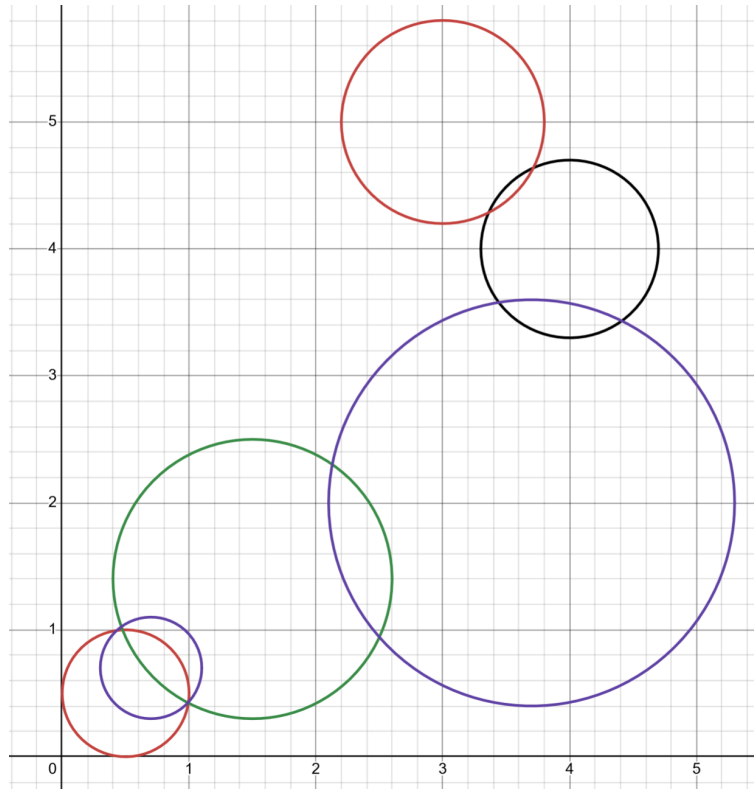


Figure 4: Given circles do form a cluster. Output = True

Collab Link: [https://colab.research.google.com/drive/1VRkZ7I0u2\\_7piJcQai-qY2zDG8bAlOa\\_?usp=sharing](https://colab.research.google.com/drive/1VRkZ7I0u2_7piJcQai-qY2zDG8bAlOa_?usp=sharing)

To begin the process of determining if circles form a cluster, I have created a function called “*circle\_struct*” that takes in a list of our c-tuple’s.

First I create an empty adjacency list that I call ‘graph’.

```
n = len(circles)
graph = {x: [] for x in range(n)} # Create an empty adjacency list
```

I then build the graph while also checking for overlaps. By building in this manner, I can ensure that we only compare each pair of circles once.

```
# Build the graph: connect nodes (circles) if they overlap
for k in range(n):
    for j in range(k + 1, n):
```

Within the loops that build the graph, I call the function ‘*compare\_adjacent\_circles*.’ This function takes in two circles to compare. If the circles overlap, we add them to each other’s adjacency lists.

```
if compare_adjacent_circles(circles[k], circles[j]):
    graph[k].append(j)
    graph[j].append(k)
```

I then check if any circle(s) are disconnected or if there are any disconnected clusters. To do this, I create a set that contains all visited circles and run a depth-first search, starting at circle 0.

```
# Create a set of all visited nodes to check for non-clustered circles or disconnected clusters
visited = set()

# Begin a depth first search to check for clusters
dfs(0, graph, visited)

# Less nodes visited than total nodes = some circles not clustered (or multiple disconnected clusters)
if len(visited) < n:
    return False

return True
```

After the DFS search, I check if the total number of circles visited is less than the total number of circles. If it is less than that, I just return False because that would mean there are circles that do not overlap or there are disconnected clusters.

The other function for this problem, 'compare\_adjacent\_circles', I take in two circles and do some math to determine if they overlap.

I calculate the distance from two circle centers with the Euclidean distance formula, the absolute difference, and the sum of both radii. I then check if they overlap or are tangent:

**$\text{abs\_diff} < \text{distance} \leq \text{rad\_sum}$**

```
def compare_adjacent_circles(circle_a, circle_b):
    x1, y1, r1 = circle_a
    x2, y2, r2 = circle_b

    # Distance between the two circles centers
    distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

    # absolute difference
    abs_diff = abs(r1 - r2)

    # Sum of radii
    rad_sum = r1 + r2

    # Check if circles overlap
    if abs_diff < distance <= rad_sum:
        return True

    return False
```

### Test Case 1:

```
# Test Case - 1 [TRUE]
case1 = [(1, 3, 0.7), (2, 3, 0.4), (3, 3, 0.9)]
sol_case1 = circle_struct(case1)
print(f"Test Case 1:")
print(f"  Circles: {case1}")
print(f"  Output: {sol_case1}")
```

*Output:*

```
Test Case 1:
  Circles: [(1, 3, 0.7), (2, 3, 0.4), (3, 3, 0.9)]
  Output: True
```

### Test Case 2:

```
# Test Case - 2 [FALSE]
case2 = [(1.5, 1.5, 1.3), (4, 4, 0.7)]
sol_case2 = circle_struct(case2)
print(f"\nTest Case 2:")
print(f"  Circles: {case2}")
print(f"  Output: {sol_case2}")
```

*Output:*

```
Test Case 2:
  Circles: [(1.5, 1.5, 1.3), (4, 4, 0.7)]
  Output: False
```

### Test Case 3:

```
# Test Case - 3 [FALSE]
case3 = [(0.5, 0.5, 0.5), (1.5, 1.5, 1.1), (0.7, 0.7, 0.4), (4, 4, 0.7)]
sol_case3 = circle_struct(case3)
print(f"\nTest Case 3:")
print(f"  Circles: {case3}")
print(f"  Output: {sol_case3}")
```

*Output:*

```
Test Case 3:
  Circles: [(0.5, 0.5, 0.5), (1.5, 1.5, 1.1), (0.7, 0.7, 0.4), (4, 4, 0.7)]
  Output: False
```

### Test Case 4:

```
# Test Case - 4 [TRUE]
case4 = [(0.5, 0.5, 0.5), (1.5, 1.5, 1.1), (0.7, 0.7, 0.4), (4, 4, 0.7), (3, 5, 0.8), (3.7, 2, 1.6)]
sol_case4 = circle_struct(case4)
print(f"\nTest Case 4:")
print(f"  Circles: {case4}")
print(f"  Output: {sol_case4}")
```

### *Output:*

```
Test Case 4:
  Circles: [(0.5, 0.5, 0.5), (1.5, 1.5, 1.1), (0.7, 0.7, 0.4), (4, 4, 0.7), (3, 5, 0.8), (3.7, 2, 1.6)]
  Output: True
```